UNIVERSIDAD NACIONAL DE COLOMBIA

FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS E INDUSTRIAL
LENGUAJES DE PROGRAMACIÓN

TAREA # 27 — TRADUCTOR DE LENGUAJE ENSAMBLADOR DE CACAOCORE-64 A LA MÁQUINA DE LA TAREA # 10 (ENIGMA-64) CONSTRUIDO CON LEX

AUTORES:

Juan Sebastian Umaña Camacho
Tomas Felipe Garzón Gómez
Michael Stiven Betancourt Gelves
Deibyd Santiago Barragán Gaitán
Maicol Sebastian Olarte Ramirez
Juan Luis Vergara Novoa
Alejandro Argüello Muñoz

Docente: Jorge Eduardo Ortiz Triviño

Bogotá D.C., Colombia — Octubre de 2026

CONTENIDO

Sección	Título	
—	Enunciado y equipo de trabajo	2
1	Marco teórico	3
2	Descripción y justificación del problema	4
3	Diseño de la solución (bloques, flujo, clases y especificación LEX)	5
4	Reglas de traducción por módulo	8
5	Los tres algoritmos clásicos en CacaoScript	10
6	Manual técnico	11
7	Manual de usuario	12
8	Experimentación y análisis de resultados	13
9	Limitaciones conocidas	15
10	Conclusiones	16
—	Referencias	16
A	Anexo: inventario del código fuente	18

Correspondencia con los entregables de una tarea automatizada. La tarea se entrega en un único archivo; la tabla indica dónde está cada entregable mínimo exigido:

Entregable exigido	Ubicación en este documento
a. Marco teórico	Sección 1
b. Descripción y justificación del problema	Sección 2
c. Diseño de la aplicación (diagramas de bloques, de flujo y UML)	Secciones 3 y 4 (Diagramas 1 a 4)
d. Código fuente completo y versiones de las librerías	Sección 6.2 (versiones) y Anexo A (inventario); el código completo acompaña la entrega en `spl/traductor/`
e. Manual de usuario y manual técnico	Secciones 6 y 7
f. Experimentación y análisis de al menos 3 escenarios	Sección 8 (Euclides, Factorial y Fibonacci, más la suite de regresión)
Información de todos los autores	Portada y tabla de roles del equipo

ENUNCIADO

1. Análisis, diseño e implementación de un Traductor de programas escritos en lenguaje ensamblador de la máquina CacaoCore64 a programas equivalentes de la máquina que su grupo de trabajo construyó como solución de la Tarea 10. Este traductor debe construirse empleando el metalenguaje LEX.

2. Escriba y pruebe el funcionamiento de tres (3) algoritmos clásicos (entre ellos incluya el algoritmo de Euclides) en el lenguaje de alto nivel de CacaoCore64. Para cada uno de ellos emplee el código en lenguaje ensamblador generado por CacaoCore64 como entrada a su traductor y verifique que, efectivamente, el código en lenguaje ensamblador obtenido para su máquina por el traductor construido corre adecuadamente en su computador. Analice los resultados obtenidos.

Datos de la tarea: inicio el 17 de octubre de 2026, entrega el 29 de octubre de 2026; categoría automatizada,

obligatoria, clase 14; grupo de Taller-Proyecto de 7 estudiantes.

Equipo de trabajo y roles

El grupo es el mismo de las Tareas # 09, # 10 y del Taller # 01. Cada integrante conservó el subsistema de Enigma-64 del que era responsable y construyó la parte del traductor que lo toca:

Int.	Nombre	Módulo	Responsabilidad en la Tarea 27
1	Juan Sebastian Umaña Camacho	`memoria.py`	Reglas LEX de memoria: `LOAD`, `STORE`, `LDB`, `STB`, `LEA`; expansión de direcciones al rango de usuario con `ADDI+SHL+ADDI` y alineación a 8 bytes.
2	Tomas Felipe Garzón Gómez	`registros.py`, `alu.py`	Asignación de los 16 registros de CacaoCore a Enigma-64 con derrame a memoria y reglas LEX de la ALU de acumulador.
3	Michael Stiven Betancourt Gelves	`control.py`	Reglas LEX de etiquetas, saltos, pila, subrutinas (`CALL`/`RET`) y `HLT`.
4	Juan Luis Vergara Novoa	`directivas.py`	Secciones, directivas y literales; normalización a 64 bits; enlace con `CargadorEnigma` y formato `.e64`.
5	Maicol Sebastian Olarte Ramirez	`enigma64/ui/`	Pestaña *Traductor* de la GUI Noctua: vista lado a lado, envío al cargador y ejecución.
6	Deibyd Santiago Barragán Gaitán	`cacaocore_src/`, `mmio_soporte.py`	Los 3 algoritmos en CacaoScript, compilación con el compilador oficial de CacaoCore y salida por MMIO.
7	Alejandro Argüello Muñoz	`verificacion.py`, `tests/`	Pipeline de verificación y contraste automatizado contra los resultados oficiales de la Tarea 10.

1. MARCO TEÓRICO

1.1 Traductores de lenguaje ensamblador

Un *traductor* es un programa que recibe un texto escrito en un lenguaje fuente y produce un texto equivalente en un lenguaje objeto. Cuando ambos lenguajes son ensambladores de máquinas distintas se habla de *traducción fuente a fuente* o *transpilación de ensamblador*; es la forma estática de la *traducción binaria*, que reemplaza la emulación instrucción por instrucción en tiempo de ejecución por una reescritura previa del programa. La traducción es correcta si, para toda entrada, el programa traducido deja en los estados observables (memoria de datos, registros visibles y banderas que se consultan) los mismos valores que el original. En este trabajo esa propiedad se comprueba experimentalmente ejecutando ambos lados.

1.2 Análisis léxico, expresiones regulares y LEX

El análisis léxico divide la entrada en *lexemas* y les asigna una *categoría* (token). Cada categoría se describe con una expresión regular; por el teorema de Kleene, toda expresión regular equivale a un autómata finito, y la construcción de Thompson seguida de la de subconjuntos produce un autómata finito determinista (AFD) que reconoce todas las categorías a la vez [1]. LEX [2] automatiza esa construcción: el programador escribe una especificación en tres partes, *definiciones*, *reglas* (patrón → acción) y *código de usuario*, y la herramienta genera el reconocedor. Ante varias coincidencias LEX elige la *más larga* y, a igual longitud, la regla escrita primero.

La idea que hace posible un traductor con LEX es que las *acciones* pueden hacer algo más que devolver un token: pueden *emitir directamente el texto traducido*. Como el ensamblador es un lenguaje de una instrucción por línea, sin anidamiento, cada forma de instrucción (mnemónico + modo de direccionamiento) se describe con una expresión regular y su acción produce la secuencia equivalente en la máquina destino. Se trabajó con **PLY 3.11** (`ply.lex`) [3], la implementación de LEX en Python que admite el enunciado: cada regla es un método `t_NOMBRE`

cuyo *docstring* es el patrón y cuyo cuerpo es la acción. PLY ordena las reglas definidas como funciones según su orden de aparición, lo que da a los patrones más específicos prioridad sobre los generales. Para expresar patrones compuestos se usa el decorador `@TOKEN`, que permite construir una regla a partir de definiciones regulares con nombre, igual que la sección de definiciones de LEX.

1.3 Las dos máquinas

CacaoCore-64 es la máquina académica de referencia del curso: 16 registros de 64 bits (`R0`–`R15`, con `R13` = SP, `R14` = LR y `R15` = ACC), una ALU *de acumulador* (toda operación deja además su resultado en R15), formato de dos operandos, memoria plana de 1 MB direccionable por bytes, ordenamiento *little-endian* y un lenguaje de alto nivel propio (*CacaoScript*) con su compilador. **Enigma-64** es la máquina diseñada por el grupo en la Tarea # 09 e implementada en la Tarea # 10: arquitectura Von Neumann multiciclo, 8 registros de propósito general (`R0` cableado a cero, `R6` = SP, `R7` = BP) más PC y SR, formato de tres operandos, inmediatos de 16 bits con extensión de signo, memoria de 4 GiB *big-endian* con palabras alineadas a 8 bytes y protección de la zona de sistema por debajo de `0x00200000`.

1.4 Asignación de registros y análisis de flujo de datos

Cuando la máquina destino tiene menos registros que la fuente hay que decidir cuáles viven en registros y cuáles se *derraman* (*spill*) a memoria. La heurística clásica [1] estima el costo de derramar un registro por su número de usos, ponderado por la profundidad de bucle (10^d), y deja en registros los de mayor peso. Del mismo modo, un *análisis de vivacidad* recorre el grafo de flujo de control para saber si un valor (aquí, las banderas Z, N, C, V) será leído antes de ser sobrescrito; solo entonces la traducción debe preservarlo. Ambas técnicas se usan en el traductor para que el código generado sea correcto sin sacrificar más eficiencia de la necesaria.

2. DESCRIPCIÓN Y JUSTIFICACIÓN DEL PROBLEMA

2.1 Descripción

Se quiere que un programa escrito en CacaoScript, compilado por el compilador oficial de CacaoCore-64, se ejecute en el computador Enigma-64 de la Tarea # 10 sin modificar ni el programa ni el compilador. La entrada del traductor es, por exigencia del enunciado, el *ensamblador* de CacaoCore; la salida es ensamblador de Enigma-64 que el ensamblador, el cargador y la CPU de la Tarea # 10 aceptan sin cambios. Las dos máquinas difieren en casi todas las dimensiones de la arquitectura, y cada diferencia es una regla que el traductor tiene que resolver:

Dimensión	CacaoCore-64	Enigma-64	Consecuencia para el traductor
Registros	16 (R0–R15); R0 escribible	7 escribibles; R0 = 0 fijo	Asignación por uso y derrame a memoria (Integrante 2)
Modelo de ALU	Acumulador, 2 operandos; resultado también en R15	3 operandos, sin acumulador	Formas implícitas se vuelven explícitas; ACC solo si se lee
Inmediatos	Hasta 64 bits en la instrucción	16 bits con signo	Constantes y direcciones con `ADDI`/`SHL`
Operandos en memoria	`ADD R3, [b]` directo	Solo `LOAD`/`STORE` (carga-almacenamiento)	Cada acceso pasa por un temporal (Integrante 1)
Memoria	1 MB plano, sin protección	4 GiB; usuario desde `0x00200000`	Código en `0x00200000`, datos en `0x00201000`
Alineación	Por bytes	Palabras de 8 bytes alineadas	Variables en múltiplos de 8 (Integrantes 1 y 4)

Endianness	Little-endian	Big-endian	Datos normalizados a palabras de 64 bits (Integrante 4)
Saltos	`JE JNE JG JL JGE JLE JS JO...`	`JZ JNZ JN JP JC JV...`	Tabla de equivalencias sobre Z y N (Integrante 3)
Directivas	`.data`, `nombre : valor`	No tiene directivas de datos	Prólogo con `STORE` o precarga del cargador
E/S	`INTR 0` (servicio de consola)	MMIO en `0xFF001000`	Aviso y rutina MMIO opcional (Integrante 6)

2.2 Justificación

Había tres caminos. (i) *Emular CacaoCore sobre Enigma-64*: un intérprete escrito en ensamblador de Enigma-64 que decodifique cada instrucción ajena; es costoso (decenas de instrucciones por cada instrucción emulada) y no aprovecha la ALU nativa. (ii) *Cambiar el generador de código del compilador de CacaoCore* para que emita Enigma-64; contradice el enunciado, que exige partir del ensamblador generado. (iii) *Traducción fuente a fuente con LEX*, el camino elegido. Sus ventajas son concretas: la salida es ensamblador legible, anotado línea a línea con la instrucción de origen, que se ensambla y ejecuta a velocidad nativa en la CPU de la Tarea # 10; las reglas LEX separan el problema en módulos independientes (memoria, ALU, control, directivas), lo que permitió repartir el trabajo entre los siete integrantes; y cada regla es pequeña, verificable y documentable. Además, el ejercicio prueba la Tarea # 10 en condiciones reales: los programas traducidos usan instrucciones, modos de direccionamiento y zonas de memoria que los programas escritos a mano no ejercitaban.

3. DISEÑO DE LA SOLUCIÓN

3.1 Arquitectura general (diagrama de bloques)

El Diagrama 1 muestra el flujo completo. Los bloques naranjas pertenecen a CacaoCore-64 (se usan sin modificar), los azules son el traductor construido en esta tarea y los verdes son los subsistemas de Enigma-64 de la Tarea # 10.

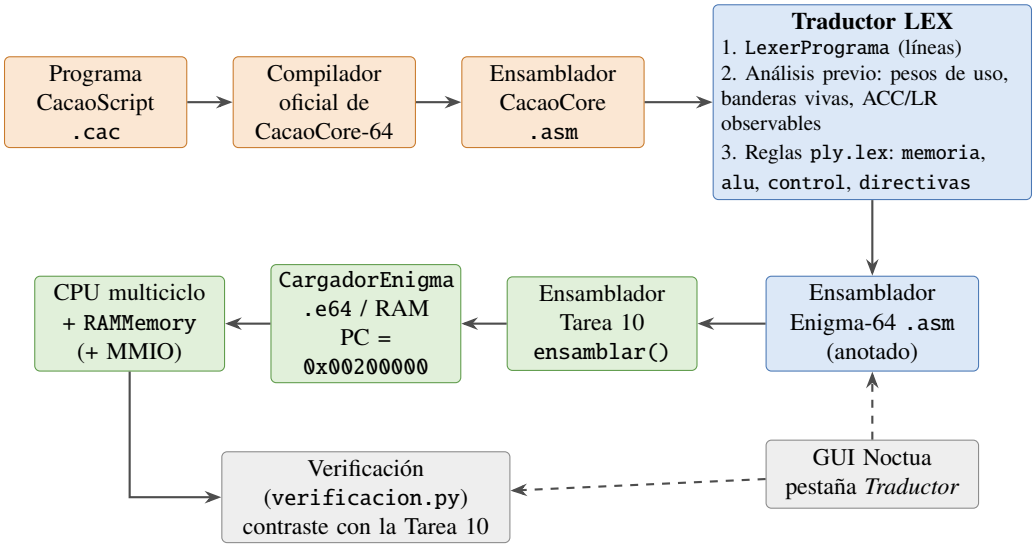


Diagrama 1: Arquitectura del traductor CacaoCore-64 → Enigma-64 y su integración con la Tarea # 10.

El traductor tiene dos niveles de LEX. El primero, `LexerPrograma` en `traductor.py`, reconoce la estructura del archivo con cuatro categorías (`SECCION`, `DATO`, `ETIQUETA`, `INSTRUCCION`) y descarta los comentarios. El segundo

nivel está formado por los analizadores de cada módulo, que reconocen la *forma* de cada instrucción y emiten su traducción. Entre ambos niveles hay un análisis global del programa, necesario porque la asignación de registros y la preservación de banderas no se pueden decidir mirando una sola línea.

3.2 Diagrama de flujo del traductor

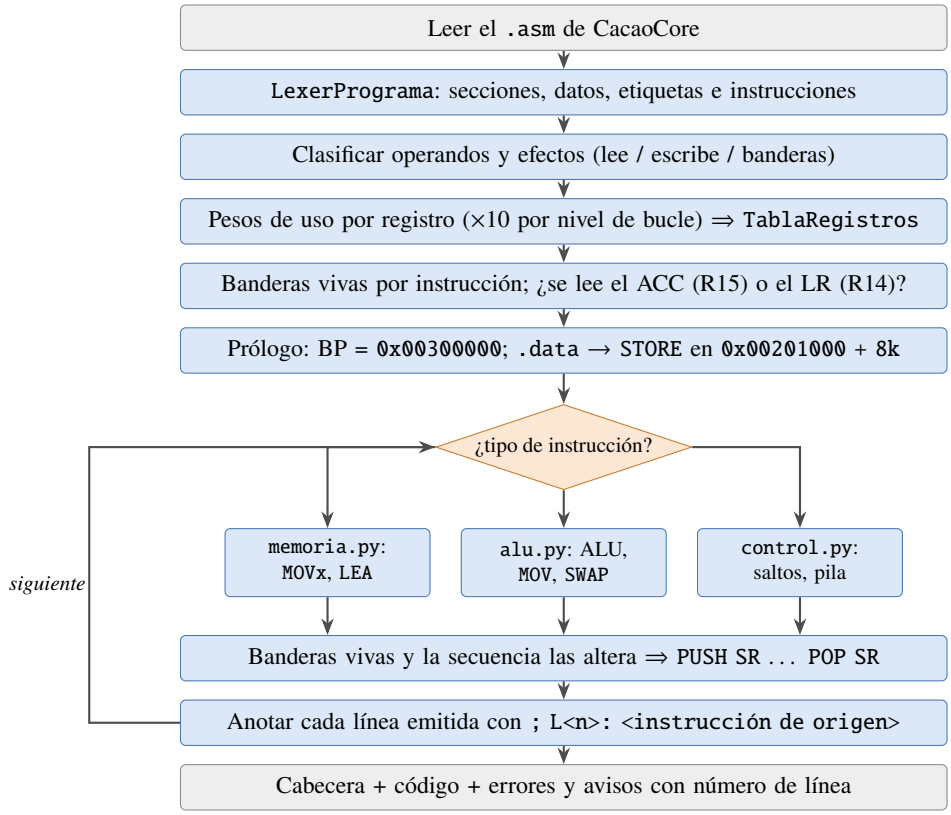


Diagrama 2: Flujo de la traducción de un programa completo (`TraductorCacaoEnigma.traducir`).

3.3 Diagrama de clases (UML)

Cada módulo expone una clase que encapsula un analizador `ply.lex` (Diagrama 3); la clase `TraductorCacaoEnigma` los orquesta y comparte con todos la misma `TablaRegistros`.

3.4 Especificación LEX

La sección de *definiciones* de `alu.py` reutiliza las mismas expresiones regulares del ensamblador de CacaoCore (`asm_lexer.py`), de modo que el traductor acepta exactamente el lenguaje que acepta la máquina de origen. A partir de ellas se arman, con `@TOKEN`, las reglas de cada modo de direccionamiento:

```
REGISTRO  = r"[Rr](?:1[0-5]|[0-9])(?![A-Za-z0-9_])"   # (R|r)(1[0-5]|[0-9])
NUMERO    = r"[+-]?\d+(?![\d.xX])"
MEMORIA   = r"0x[0-9A-Fa-f]+"                          # direccion directa
VARIABLE  = r"[a-z_][a-z0-9_]*"
DIRECTO   = MEMORIA | \[VARIABLE\] | \[MEMORIA\]
INDIRECTO = \[REGISTRO\]
FIN       = r"(?=[ \t]*(?:[;#]|$))"                    # nada mas en la instruccion
```

El ancla `FIN` hace las formas mutuamente excluyentes (`ADD R1` no reconoce el prefijo de `ADD R1, R2`). Las

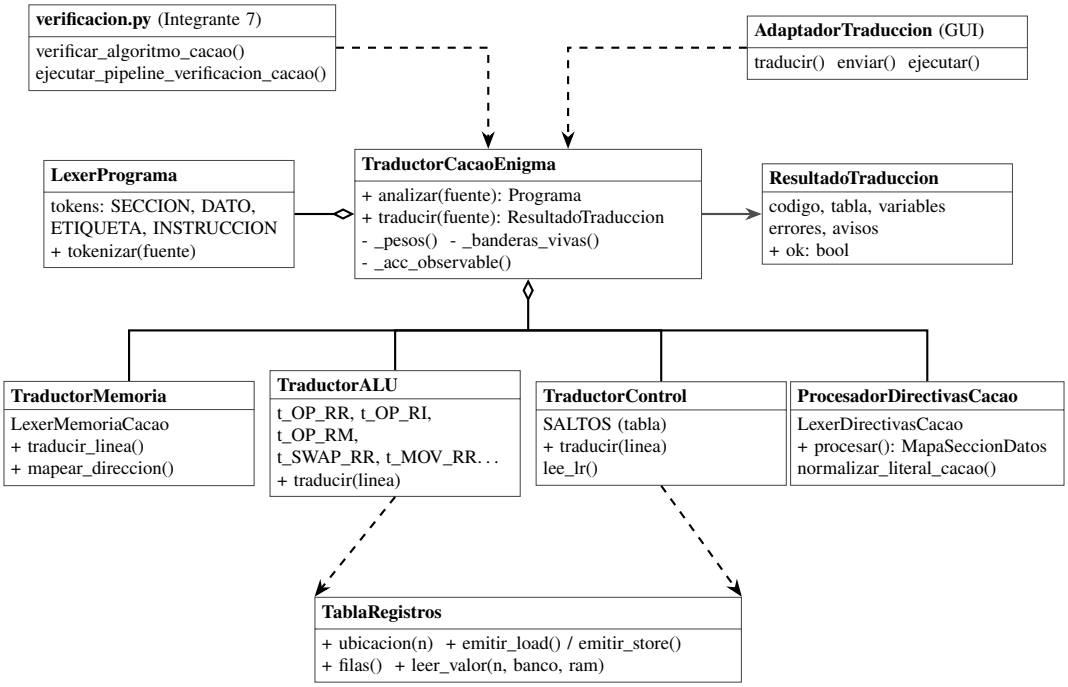


Diagrama 3: Clases principales del paquete `spl.traductor` y sus clientes. Rombo: composición (el traductor contiene los analizadores); flecha discontinua: dependencia. Los clientes (arriba) usan el traductor a través de `traducir_texto` y `ejecutar_en_enigma`.

reglas de `alu.py` son `t_OP_RR`, `t_OP_RI`, `t_OP_RM`, `t_OP_RN`, `t_OP_R`, `t_OP_I`, `t_OP_M`, `t_UNARIA_R`, `t_UNARIA_M`, `t_DESP_I`, `t_DESP_RI`, `t_SWAP_R`, `t_SWAP_RR`, `t_MOV_RR`, `t_MOV_RI`, `t_LOAD_I`, `t_STORE_R` y `t_FLOTANTE`; las de `control.py`, `ETIQUETA`, `SALTO_INCONDICIONAL`, `SALTO_CONDICIONAL`, `SALTO_ABSOLUTO`, `SALTO_FPU`, `LLAMADA`, `RETORNO`, `PILA`, `PARADA` y `NO_CACAO`. Un ejemplo de regla completa, con su acción traductora:

```
@TOKEN(rf"(?:{_M_CONDICIONALES}){_ESP1}{DESTINO}{FIN}")
def t_SALTO_CONDICIONAL(self, t):
    mnemonico, destino = t.value.split(None, 1)
    for salto in SALTOS[mnemonico.lower()]:      # p. ej. JGE -> ("JP", "JZ")
        self._codigo.append(f"{salto} {destino.strip()}")
```

`t_error` trata los errores (registro inválido, modo de direccionamiento no soportado, mnemónico desconocido o con mayúsculas mezcladas): el mensaje queda asociado a su número de línea y la traducción continúa con la línea siguiente, de modo que un solo pase informa todos los errores del programa.

3.5 Mapa de memoria y convenciones

Zona de Enigma-64	Uso en el código traducido
`0x00000000`–`0x001FFFFF`	Sistema (vectores, monitor); protegida en modo usuario, nunca se toca
`0x00200000`	Código traducido (punto de entrada; hasta 4 KiB)
`0x00201000 + 8k`	Variables de `.data`, en el orden de declaración, alineadas a 8 bytes
`0x00300000 + 8k`	Área de derrame de registros, direccionada como `[R7 + 8k]`
`0xEFFFFFFF` hacia abajo	Pila compartida (R6 = SP de Enigma = R13 de CacaoCore)
`0xFF001010`	Registro `DATA` de la pantalla MMIO (salida opcional)

4. REGLAS DE TRADUCCIÓN POR MÓDULO

4.1 Registros (Integrante 2)

`R0` de Enigma-64 nunca recibe un registro de CacaoCore (perdería las escrituras); en cambio se usa como operando cero. Los tres registros de mayor peso van a `R1`–`R3` (cuatro, usando también `R7`, si no hace falta derramar), `R4` y `R5` son temporales (TMP1, TMP2), `R6` recibe al SP (`R13`) y el resto se derrama a ranuras `[R7 + 8k]`. Con base en registro, cada derrame es un solo `LOAD` o `STORE`, instrucciones que no alteran banderas. El traductor imprime la tabla calculada como cabecera del archivo generado; para Fibonacci es:

```
; Asignacion de registros:
;   R0 -> R3     R2 -> R2     R3 -> R1     R13 (SP) -> R6
;   R4 -> derrame [R7]        R5 -> derrame [R7 + 8]    R10 -> derrame [R7 + 16]
;   R11 -> derrame [R7 + 24]  R12 -> derrame [R7 + 32]
```

4.2 Memoria y direccionamiento (Integrante 1)

Enigma-64 es de carga-almacenamiento y sus inmediatos son de 16 bits, así que una dirección de usuario se construye con `ADDI+SHL+ADDI` en el temporal `R5` antes de cada acceso. Los mnemónicos de tamaño byte (`MOVB`, `LOADB`, `STOREB`) se traducen a `LDB`/`STB`; los de palabra a `LOAD`/`STORE`. Las direcciones absolutas de CacaoCore se reubican por encima de `0x00200000` y los desplazamientos se ajustan a múltiplos de 8 para no activar `STATUS_MISALIGNED`.

CacaoCore	Enigma-64	Nota
`MOVD R0, [a]`	`ADDI R5, R0, 0x0020; SHL R5, R5, 16; ADDI R5, R5, 0x1000; LOAD R2, [R5]`	a en `0x00201000`; R0→R2
`MOVD [a], R3`	misma dirección en R5; `STORE R1, [R5]`	
`MOVD R4, [R3]`	`LOAD R4, [R1]`	Indirecto por registro
`LEA R4, [fib]`	dirección en R4; `STORE R4, [R7]` si R4 está derramado	No toca banderas
`MOVB R1, [x]`	dirección en R5; `LDB R1, [R5]`	Acceso de un byte

4.3 ALU de acumulador (Integrante 2)

Las operaciones de dos operandos pasan a formato de tres (`ADD Rx, Ry` → `ADD x, x, y`); las que tienen operando en memoria cargan antes el valor en TMP2; las inmediatas que no caben en 16 bits se construyen con `cargar_constante`. La traducción respeta la semántica real del emulador de CacaoCore, que se revisó en su código fuente:

CacaoCore	Enigma-64	Justificación
`SUB R3, [b]`	dirección en R5; `LOAD R5, [R5]`; `SUB x, x, R5`	No hay operandos en memoria en la ALU de Enigma
`MUL R3, 8`	`ADDI R5, R0, 8`; `MUL x, x, R5`	Enigma no tiene `MULI`
`DIV Rx, Ry`	`DIV T1, x, y; MUL T2, T1, y; SUB y, x, T2; ADD x, T1, R0`	CacaoCore deja el residuo en Ry; se recalcula como $x - (x/y)\,y$
`CMPZ Rx`	`CMP R0, x`	Banderas de $0 - R_x$
`SHR Rx, n`	`ASR x, x, n`	El SHR de CacaoCore conserva el signo
`NEG Rx`	`SUB x, R0, x`	Aprovecha R0 = 0
`MOVD Rx, Ry`	`ADD x, y, R0` (o `PUSH y; POP x` si hay banderas vivas)	MOV no altera banderas en CacaoCore
`SWAP Rx, Ry`	`PUSH x; PUSH y; POP x; POP y`	Sin temporal ni banderas; válido con Rx = Ry
`ROL Rx, n`	`SHR T2, x, 64-n; SHL x, x, n; OR x, x, T2`	Rotación de 64 bits

El efecto lateral *ACC = resultado* solo se emite si alguna instrucción del programa lee R15; en los tres algoritmos no ocurre, así que el ACC ni siquiera ocupa un registro. Si una instrucción que en CacaoCore no altera banderas (`MOV`, `LEA`, accesos a memoria) se expande en código que sí lo hace (`ADDI`/`SHL`) y las banderas están vivas, la secuencia se envuelve en `PUSH SR` . . . `POP SR`.

4.4 Control de flujo, pila y subrutinas (Integrante 3)

Los saltos condicionales de comparación de CacaoCore solo consultan N y Z, y como Z = 1 implica N = 0 se obtienen las equivalencias de la tabla. `PUSH`/`POP` se traducen uno a uno porque el SP de CacaoCore vive en el SP de Enigma-64 y ambas pilas crecen hacia abajo en palabras de 8 bytes; `CALL`/`RET` también, salvo que el programa lea `LR` (R14), caso en el que se emite un trampolín que reproduce la escritura de LR.

`JE→JZ`	`JNE→JNZ`	`JG→JP`	`JL→JN`	`JGE→JP+JZ`	`JLE→JN+JZ`	`JS→JN`	`JO→JV`

Las instrucciones de Enigma-64 que CacaoCore no tiene (`JN`, `JP`, `JMPR`, `ENTER`, `LEAVE`. . .) se reconocen solo para emitir un error que indica la forma correcta en CacaoCore.

4.5 Directivas, literales y enlace con el cargador (Integrante 4)

`LexerDirectivasCacao` reconoce secciones (`.data`, `.text`, `[DATA]`, `[CODE]`), directivas (`.word`, `.quad`, `.byte`, `.ascii`, `.space`, `.align`) y literales decimales, hexadecimales, binarios, flotantes IEEE 754, caracteres y cadenas. `normalizar_literal_cacao` lleva todos a una palabra de 64 bits en complemento a dos (`-1` → `0xFFFFFFFFFFFFFFFF`). Cada variable recibe la dirección `0x00201000 + 8k`. Hay dos maneras de cargar los datos: un *prólogo* de `STORE` dentro del propio código traducido (autocontenido) o la *precarga* en `RAMMemory` por medio de `CargadorEnigma`, que además puede exportar un ejecutable `.e64` (cabecera `b'ENIG'`, tabla de símbolos y punto de entrada `0x00200000`) que la GUI de la Tarea # 10 abre directamente.

4.6 Entrada/salida por MMIO (Integrante 6)

El compilador de CacaoCore traduce `show x` a `MOVH R10, 0; LEA R11, [x]; MOVW R12, 1; INTR 0`. Enigma-64 no tiene interrupciones de software de E/S, así que `INTR 0` produce un *aviso* y el resultado queda en su variable de RAM. Opcionalmente, `mmio_soporte.py` añade antes de `HLT` una rutina que convierte el valor a ASCII (divisiones sucesivas por 10 y la pila para invertir los dígitos) y lo escribe en `0xFF001010`, el registro `DATA` de la pantalla MMIO.

5. LOS TRES ALGORITMOS CLÁSICOS EN CACAOSCRIPT

Los algoritmos se escribieron en el lenguaje de alto nivel de CacaoCore (`spl/traductor/cacaocore_src/`) con los mismos datos de los programas oficiales de la Tarea # 10, para poder contrastar resultados. El script `tools/compilar_algoritmos_cacao.py` invoca el compilador oficial de CacaoCore (`Compiler.compile_full`) y deja los tres `.asm` en `spl/traductor/ejemplos/`.

Euclides (obligatorio)

```
let a: int = 48
let b: int = 18
asLongAs a != b {
  if a > b {
    set a = a - b
  }
  otherwise {
    set b = b - a
  }
}
show a
```

Factorial

```
let n: int = 5
let fact: int = 1
asLongAs n > 1 {
  set fact = fact * n
  set n = n - 1
}
show fact
```

Fibonacci

```
let fib: int[7] =
    0, 1, 0, 0, 0, 0, 0
let i: int = 2
asLongAs i < 7 {
  set fib[i] = fib[i - 1]
             + fib[i - 2]
  set i = i + 1
}
let res: int = fib[6]
show res
```

5.1 Del ensamblador de CacaoCore al de Enigma-64: Euclides

El compilador de CacaoCore materializa cada condición como un booleano en un registro (0 o 1) y luego lo compara con cero. A la izquierda, el inicio del bucle generado por CacaoCore; a la derecha, la salida real del traductor (abreviada solo en la cabecera de comentarios):

```
.data
a : 48
b : 18
.text
LWHILE_START3:
MOVD R0, [a]
CMP R0, [b]
JNE LTRUE0
MOVD R0, 0
JMP LFALSE0
LTRUE0:
MOVD R0, 1
LFALSE0:
CMP R0, 0
JZ LWHILE_END3
```

```
LWHILE_START3:
    ADDI R5, R0, 0x0020       ; L12: MOVD R0, [a]
    SHL R5, R5, 16            ; L12: MOVD R0, [a]
    ADDI R5, R5, 0x1000       ; L12: MOVD R0, [a]
    LOAD R2, [R5]             ; L12: MOVD R0, [a]
    ADDI R5, R0, 0x0020       ; L13: CMP R0, [b]
    SHL R5, R5, 16            ; L13: CMP R0, [b]
    ADDI R5, R5, 0x1008       ; L13: CMP R0, [b]
    LOAD R5, [R5]             ; L13: CMP R0, [b]
    CMP R2, R5                ; L13: CMP R0, [b]
    JNZ LTRUE0                ; L14: JNE LTRUE0
    ADD R2, R0, R0            ; L15: MOVD R0, 0
    JMP LFALSE0               ; L16: JMP LFALSE0
LTRUE0:
    ADDI R2, R0, 1            ; L18: MOVD R0, 1
LFALSE0:
    CMP R2, R0                ; L20: CMP R0, 0
    JZ LWHILE_END3            ; L21: JZ LWHILE_END3
```

Se observan las reglas en acción: el R0 de CacaoCore vive en el R2 de Enigma-64, el operando en memoria `[b]` se carga en el temporal R5, `JNE` pasa a `JNZ`, `MOVD R0, 0` aprovecha el cero cableado (`ADD R2, R0, R0`) y cada línea conserva como comentario la instrucción y la línea de origen. La regla `JG LTRUE1` de la comparación `a > b` se traduce a `JP LTRUE1`.

5.2 Acceso indexado: Fibonacci

Para `fib[i - 1]` el compilador de CacaoCore escala el índice por 8, toma la dirección base con `LEA` y accede de forma indirecta. Como CacaoCore usa R4 y R5 (que en Enigma son temporales), el traductor los derrama al área

[R7]:

```
    LOAD R1, [R5]            ; L29: MOVD R3, [i]          (R5 = &i = 0x00201038)
    SUBI R1, R1, 1           ; L30: SUB R3, 1
    ADDI R5, R0, 8           ; L31: MUL R3, 8
    MUL R1, R1, R5           ; L31: MUL R3, 8
    ADDI R4, R0, 0x0020      ; L32: LEA R4, [fib]
    SHL R4, R4, 16           ; L32: LEA R4, [fib]
    ADDI R4, R4, 0x1000      ; L32: LEA R4, [fib]
    STORE R4, [R7]           ; L32: LEA R4, [fib]          (R4 de CacaoCore derramado)
    LOAD R5, [R7]            ; L33: ADD R3, R4
    ADD R1, R1, R5           ; L33: ADD R3, R4
    LOAD R4, [R1]            ; L34: MOVD R4, [R3]          (fib[i-1])
    STORE R4, [R7]           ; L34: MOVD R4, [R3]
```

6. MANUAL TÉCNICO

6.1 Estructura del paquete

Archivo	Contenido
`spl/traductor/traductor.py`	`LexerPrograma`, `TraductorCacaoEnigma` (análisis global y orquestación), `ejecutar_en_enigma`, CLI
`spl/traductor/memoria.py`	`LexerMemoriaCacao`, `TraductorMemoria`: accesos a memoria, direcciones y alineación
`spl/traductor/registros.py`	`TablaRegistros`, `mapear_registro`, derrames, `cargar_constante`
`spl/traductor/alu.py`	Definiciones regulares y reglas LEX de la ALU, `MOV`, `SWAP` y rotaciones
`spl/traductor/control.py`	`TraductorControl`: etiquetas, saltos, pila, `CALL`/`RET`, `HLT`
`spl/traductor/directivas.py`	`LexerDirectivasCacao`, literales, `MapaSeccionDatos`, `.e64` y `CargadorEnigma`
`spl/traductor/mmio_soporte.py`	Rutina de salida ASCII en `0xFF001010` y ejecución con periféricos
`spl/traductor/verificacion.py`	Pipeline de verificación y contraste con la Tarea # 10
`enigma64/ui/servicios/traduccion.py`	`AdaptadorTraduccion`: traductor + cargador + RAM + CPU para la GUI
`enigma64/ui/paneles/panel_traduccion.py`	`PanelTraduccion`: pestaña Tkinter (no importa hardware, por la regla de aislamiento)

6.2 Requisitos y versiones

El emulador y la GUI usan solo la biblioteca estándar de Python (incluido `tkinter`). Las bibliotecas fuera de la instalación estándar están en `requirements.txt`:

Componente	Versión probada	Requerida	Uso
Python	3.13.11	≥ 3.10	Lenguaje de implementación
PLY (`ply.lex`)	3.11	≥ 3.11	Metalenguaje LEX del traductor
pytest	9.2.0	≥ 8.0	Suite de pruebas
CacaoCore-64 (repositorio del curso)	—	solo para recompilar	Recompilar los `.cac` (variable `CACAOCORE_PATH`)

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

6.3 API de programación

```
from spl.traductor import traducir_texto, ejecutar_en_enigma
from spl.traductor import ejecutar_pipeline_verificacion_cacao

res = traducir_texto(open("spl/traductor/ejemplos/cacao_euclides.asm").read())
assert res.ok                       # sin errores; res.avisos trae el de INTR 0
cpu, banco, ram = ejecutar_en_enigma(res.codigo, usar_cargador=True)
mcd, _ = ram.mem_read(res.variables["a"], 8)   # 6

reporte = ejecutar_pipeline_verificacion_cacao()   # los 3 algoritmos
print(reporte.exito_total)                         # True
```

`TraductorCacaoEnigma` acepta opciones: `sincronizar_acc=True` fuerza la copia del resultado en el registro del ACC, y la dirección base de datos se puede cambiar. Para añadir una instrucción basta con escribir una regla `t_...` en el módulo correspondiente (patrón con las definiciones existentes y acción que agrega líneas a `self._codigo`) y declarar sus efectos sobre registros y banderas para que el análisis global la tenga en cuenta.

7. MANUAL DE USUARIO

7.1 Línea de comandos

```
# Traducir y mostrar en consola (los avisos y errores salen por stderr)
python -m spl.traductor spl/traductor/ejemplos/cacao_euclides.asm
# Traducir a un archivo .asm de Enigma-64
python -m spl.traductor spl/traductor/ejemplos/cacao_euclides.asm -o euclides_enigma.asm
# Comprobar que la salida ensambla (--verificar) o ejecutarla en el emulador (--ejecutar)
python -m spl.traductor spl/traductor/ejemplos/cacao_euclides.asm --ejecutar --cargador
# Exportar un ejecutable .e64 para abrirlo en la GUI de la Tarea 10
python -m spl.traductor spl/traductor/ejemplos/cacao_euclides.asm --exportar-e64 euclides.e64
# Recompilar los .cac con el compilador oficial de CacaoCore
CACAOCORE_PATH=/ruta/a/CacaoCore_64 python tools/compilar_algoritmos_cacao.py
```

Salida real de `-ejecutar` para Euclides (stderr):

```
aviso (linea 46): 'INTR 0': Enigma-64 no tiene interrupcion de software para E/S; se omite (el dato
    queda en memoria)
ensamblado correcto: 305 bytes
HLT tras 199 instrucciones (995 ciclos)
  [a] @0x00201000 = 6
  [b] @0x00201008 = 6
```

El código de salida es 0 si la traducción no tuvo errores y 1 en caso contrario. Cada error se informa como `error (linea N): mensaje`.

7.2 Interfaz gráfica (pestaña Traductor)

Se abre con `python -m enigma64.ui` (pestaña *Traductor* dentro de la ventana completa, que comparte RAM, registros y unidad de control con las demás pestañas) o, de forma aislada, con `python -m enigma64.ui.paneles.panel_traduccion`. Pasos:

1. Elegir un programa en *Programa CacaoCore* (los tres algoritmos aparecen primero) o *Abrir .asm...* desde disco. El editor de la izquierda es editable.
2. *Traducir*: el código de Enigma-64 aparece a la derecha; debajo, líneas, bytes de código máquina, variables y avisos/errores. Las líneas con error se pintan en rojo y bloquean el envío.
3. Pulsar una línea de cualquier lado resalta la instrucción de CacaoCore y todas las de Enigma-64 que generó.
4. *Enviar al Cargador / RAM*: carga el binario en `0x00200000` y los datos en `0x00201000`.
5. *Ejecutar*: corre la CPU hasta `HLT` y, para los tres algoritmos, contrasta con la Tarea # 10.
6. *Guardar .asm Enigma...*: exporta la traducción.

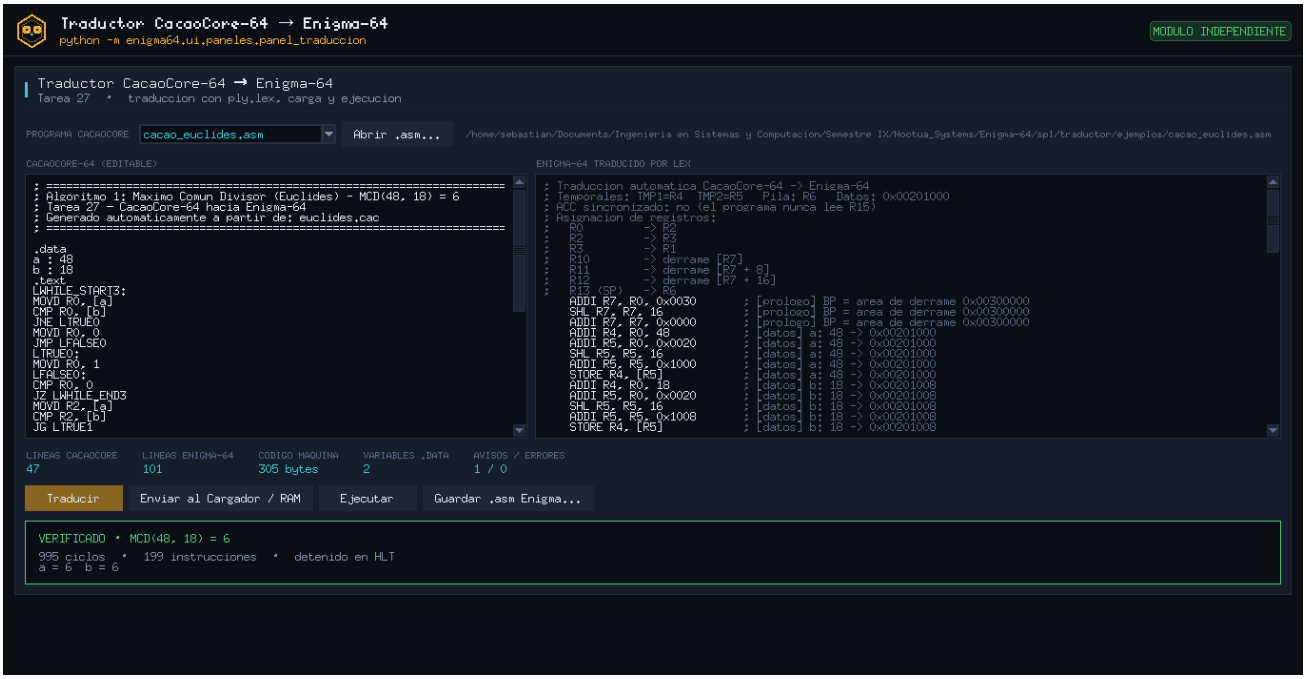


Diagrama 4: Pestaña Traductor *tras traducir, cargar y ejecutar* `cacao_euclides.asm`*:* VERIFICADO · MCD(48, 18) = 6.

8. EXPERIMENTACIÓN Y ANÁLISIS DE RESULTADOS

8.1 Metodología

Para cada algoritmo, `verificacion.py` ejecuta el pipeline completo sin intervención manual: (1) lee el `.asm` generado por el compilador de CacaoCore; (2) lo traduce con el traductor LEX; (3) lo ensambla con el ensamblador de la Tarea # 10; (4) lo carga con `CargadorEnigma` en `0x00200000`; (5) lo ejecuta en la CPU multiciclo hasta `HLT`; (6) lee las variables en RAM; (7) ejecuta el programa *nativo* oficial de la Tarea # 10 (`enigma64/programas.py`)

con los mismos datos y (8) compara ambos con el resultado esperado. Las cifras de esta sección se obtuvieron el 29 de octubre de 2026 con Python 3.13.11 y PLY 3.11.

8.2 Escenario 1: algoritmo de Euclides, MCD(48, 18)

Entrada: `a = 48` en `0x00201000`, `b = 18` en `0x00201008`. **Esperado (Tarea 10):** 6. La traducción no tuvo errores y un aviso (`INTR 0`). El programa ensambló en 305 bytes y se detuvo en `HLT` tras 199 instrucciones y 995 ciclos. Volcado final: `[a] = 6`, `[b] = 6`. La traza reproduce las restas sucesivas $(48, 18) \to (30, 18) \to (12, 18) \to (12, 6) \to (6, 6)$: cinco evaluaciones de la condición `a != b` y cuatro restas, igual que en el programa nativo. Con la rutina MMIO la pantalla muestra `6`. **Resultado: correcto.**

8.3 Escenario 2: factorial, 5!

Entrada: `n = 5`, `fact = 1`. **Esperado:** 120. Sin errores; 216 bytes; `HLT` tras 164 instrucciones y 820 ciclos. Volcado: `[fact] = 120` en `0x00201008` y `[n] = 1`, porque el bucle de CacaoScript termina cuando `n > 1` es falso. La multiplicación con operando en memoria (`MUL R2, [n]`) se tradujo a carga en el temporal más `MUL R1, R1, R5`. Salida MMIO: `120`. **Resultado: correcto.**

8.4 Escenario 3: sucesión de Fibonacci (7 términos)

Entrada: arreglo `fib` de 7 palabras en `0x00201000`–`0x00201030` con $[0, 1, 0, 0, 0, 0, 0]$ e `i = 2`. **Esperado:** $[0, 1, 1, 2, 3, 5, 8]$. Sin errores; 501 bytes; `HLT` tras 402 instrucciones y 2010 ciclos. Volcado: `fib..fib6` = $[0, 1, 1, 2, 3, 5, 8]$, `[i] = 7` y `[res] = 8` en `0x00201040`. Es el escenario más exigente: usa direccionamiento indirecto por registro, escalamiento del índice, `LEA`, y requiere derramar cinco registros de CacaoCore (R4, R5, R10, R11, R12). Salida MMIO: `8`. **Resultado: correcto.**

8.5 Resumen y comparación con los programas nativos de la Tarea 10

	Instr. estáticas		Bytes de código		Instr. ejecutadas			
Algoritmo	**Cacao**	**Traducido**	**Nativo**	**Traducido**	**Nativo**	**Traducido**	**Factor**	**Resultado**
Euclides	29	79	50	305	29	199	6,9×	6 = 6 ✓
Factorial	20	56	41	216	24	164	6,8×	120 = 120 ✓
Fibonacci	41	129	63	501	45	402	8,9×	[0..8] = [0..8] ✓

Instr. estáticas: instrucciones del archivo (sin etiquetas ni datos). En la CPU de la Tarea # 10 cada instrucción tarda 5 ciclos (FETCH, DECODE, EXECUTE, MEMORY, WRITE-BACK), de modo que los ciclos son 5× las instrucciones ejecutadas (995, 820 y 2010 frente a 145, 120 y 225).

Los tres resultados coinciden con los oficiales de la Tarea # 10, y el pipeline informa `exito_total = True`.

8.6 Análisis de resultados

Corrección. La equivalencia no se limita al valor final: la suite del Integrante 2 compara registro por registro el estado final contra un modelo de CacaoCore escrito en las pruebas (incluidos los registros derramados, que se leen de su ranura). Esto incluye Euclides con 8 pares de entradas (hasta 2^{40}), un programa con todas las operaciones de la ALU en todas sus formas y 120 programas aleatorios (60 semillas × 2 modos de ACC); en una campaña aparte con 3000 programas aleatorios más (4958 ejecuciones en los dos modos) no apareció ninguna diferencia.

¿De dónde sale el factor de 7 a 9? La expansión estática del traductor es de solo 2,7 a 3,1 instrucciones por instrucción de CacaoCore. Al clasificar las instrucciones emitidas se encuentra la causa principal:

Algoritmo	Prólogo y datos	Cuerpo	Cálculo de direcciones	Derrames	Observación
Euclides	13	66	33 (50 %)	3	11 accesos a `[a]`/`[b]` × 3 instrucciones
Factorial	13	43	21 (49 %)	2	7 accesos a variables
Fibonacci	41	88	30 (34 %)	13	9 variables en el prólogo; 5 registros derramados

Cerca de la mitad del cuerpo traducido es la secuencia `ADDI+SHL+ADDI` que construye una dirección de 32 bits con inmediatos de 16 bits; es el precio de pasar de una máquina con operandos en memoria y direcciones en la instrucción a una de carga-almacenamiento. El resto del factor no lo introduce el traductor sino el *compilador* de CacaoCore, que materializa cada condición como 0/1 y vuelve a compararla (siete instrucciones por condición donde el programa nativo usa dos) y relee las variables de memoria en cada iteración, mientras que los programas nativos de la Tarea # 10, escritos a mano, mantienen todo en registros. Fibonacci tiene el mayor factor porque su compilación usa ocho registros de CacaoCore (cinco van a memoria) y el índice se recalcula tres veces por iteración.

Mejoras posibles. Un optimizador *peephole* posterior podría reutilizar la dirección ya construida en R5 cuando dos accesos consecutivos usan la misma variable (en Euclides, `[a]` y `[b]` se construyen 11 veces) o dejar una base fija en un registro y usar `[Rb + off]`. No se hizo porque el objetivo era una traducción verificable instrucción por instrucción, con la línea de origen en cada línea emitida.

8.7 Suite de pruebas automatizadas

Archivo	Pruebas	Qué verifica
`test_traductor_memoria.py`	65	Lexer de memoria, direcciones, alineación, byte/palabra
`test_traductor_registros_alu.py`	244	Asignación, constantes, cada regla LEX, banderas, aleatorias
`test_traductor_control.py`	193	Saltos, pila, `CALL`/`RET`, recursión, errores
`test_traductor_directivas.py`	19	Directivas, literales, endianness, `.e64`, cargador
`test_algoritmos_cacao.py`	10 (+3 omitidas)	Algoritmos y MMIO (las omitidas requieren CacaoCore)
`tests/test_traductor_cacao.py`	12 (+19 subpruebas)	Pipeline completo contra la Tarea # 10
`tests/ui/test_ui_traduccion.py`	22	Servicio y pestaña de la GUI
Total traductor	**565**	**565 aprobadas, 3 omitidas, 0 fallos**

La suite completa del repositorio (emulador, FPU, SPL, GUI y traductor) también pasa: `1404 passed, 4 skipped`. Las 3 pruebas omitidas del traductor son las que recompilan los `.cac`, y solo se ejecutan si el repositorio de CacaoCore está disponible.

9. LIMITACIONES CONOCIDAS

- **División con signos distintos:** Enigma-64 trunca hacia cero y el emulador de CacaoCore usa el piso de Python; con operandos de distinto signo y residuo no nulo el cociente difiere en 1. Con operandos positivos (los tres algoritmos) coinciden.
- **Banderas C y V:** Z y N son exactas, y son las únicas que leen `JZ`, `JNZ`, `JG`, `JL`, `JGE`, `JLE`, `JS` y `JNS`. C y V pueden

diferir porque CacaoCore las limpia en las operaciones lógicas y Enigma-64 no.
- **Sin traducción:** `INTR` (aviso; salida MMIO opcional), FPU, `SEXT`, `JU`/`JNU` y saltos a direcciones absolutas producen aviso o error con número de línea.
- **Tamaño:** los datos empiezan en `0x00201000`, así que el código traducido no puede superar 4 KiB (lo comprueba `ejecutar_en_enigma`).
- **Pila compartida:** R13 de CacaoCore es el SP de Enigma-64; un programa que cargue en R13 una dirección propia de CacaoCore rompería `PUSH`/`POP`.

10. CONCLUSIONES

1. Se construyó con el metalenguaje LEX (PLY 3.11) un traductor completo de ensamblador CacaoCore-64 a ensamblador de la máquina de la Tarea # 10. Los tres algoritmos clásicos escritos en CacaoScript (Euclides, factorial y Fibonacci), compilados con el compilador oficial de CacaoCore y traducidos, corren en el emulador de Enigma-64 y producen exactamente los resultados oficiales: MCD(48, 18) = 6, 5! = 120 y [0, 1, 1, 2, 3, 5, 8].

2. La forma de LEX encaja con el problema: como el ensamblador no tiene anidamiento, una expresión regular por modo de direccionamiento y una acción que emite la secuencia equivalente bastan para traducir cada instrucción. Reutilizar las expresiones regulares del ensamblador de CacaoCore garantiza que se acepta exactamente su lenguaje.

3. LEX resuelve lo local, pero la corrección exigió un análisis global previo: asignar 16 registros a 7 según su uso, saber cuándo las banderas o el ACC son observables y proteger solo esos casos. Separar *qué* se reconoce (patrones) de *qué* se emite (acciones) y de *qué* se sabe del programa (análisis) fue la decisión de diseño más importante.

4. La traducción tiene un costo medible de 6,8 a 8,9 veces más instrucciones ejecutadas que los programas escritos a mano para Enigma-64. Cerca de la mitad lo causa la construcción de direcciones con inmediatos de 16 bits, y el resto viene del compilador de CacaoCore. Hacer la traducción línea a línea y anotada la hizo verificable a cambio de eficiencia, y hace posible un optimizador posterior.

5. El trabajo también validó la Tarea # 10: el ensamblador, el cargador, la protección de memoria, la pila y la CPU ejecutaron sin cambios código generado automáticamente que nadie del grupo escribió. Y sí: resulta notable que unas cuantas decenas de expresiones regulares con sus acciones basten para que un programa pensado para una máquina corra en otra completamente distinta. No es magia, es la teoría de autómatas convertida en una herramienta de ingeniería.

REFERENCIAS

[1] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2.ª ed. Boston: Addison-Wesley, 2006.

[2] M. E. Lesk y E. Schmidt, "Lex — A Lexical Analyzer Generator", Bell Laboratories, Computing Science Technical Report 39, 1975.

[3] D. M. Beazley, "PLY (Python Lex-Yacc)", versión 3.11, 2018. [En línea]. Disponible: `https://www.dabeaz.com/ply/`

[4] J. Levine, *flex & bison*. Sebastopol: O'Reilly Media, 2009.

[5] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5.ª ed. Morgan Kaufmann, 2014.

[6] Curso de Lenguajes de Programación, “LP02 — CacaoCore_64: emulador, ensamblador y compilador”, Universidad Nacional de Colombia, 2026.

[7] Noctua Systems, “Reporte técnico de implementación: Tarea 10 — Enigma-64”, Universidad Nacional de Colombia, 2026.

ANEXO A: INVENTARIO DEL CÓDIGO FUENTE

Archivo	Líneas	Autor
`spl/traductor/traductor.py`	599	Integrantes 2, 3 y 4 (orquestación)
`spl/traductor/memoria.py`	660	Integrante 1
`spl/traductor/registros.py`	354	Integrante 2
`spl/traductor/alu.py`	787	Integrante 2
`spl/traductor/control.py`	376	Integrante 3
`spl/traductor/directivas.py`	606	Integrante 4
`spl/traductor/mmio_soporte.py`	148	Integrante 6
`spl/traductor/verificacion.py`	372	Integrante 7
`spl/traductor/__init__.py`, `__main__.py`	87	Todos
`spl/traductor/cacaocore_src/*.cac` (3) y `ejemplos/*.asm` (11)	—	Integrantes 2 y 6
`tools/compilar_algoritmos_cacao.py`	142	Integrante 6
`enigma64/ui/servicios/traduccion.py`	212	Integrante 5
`enigma64/ui/paneles/panel_traduccion.py`	363	Integrante 5
`tests/test_spl/traductor/` (5 archivos)	2155	Integrantes 1, 2, 3, 4 y 6
`tests/test_traductor_cacao.py`	252	Integrante 7
`tests/ui/test_ui_traduccion.py`	228	Integrante 5
Total	**7341**	